

Setting	8, 512	512, 8	8,8
13B tuned	16.35 _{0.69}	0.33 _{0.02}	0.26 _{0.01}
13B proxy-tuned	41.55 _{1.50}	0.76 _{0.02}	0.63 _{0.03}
Slowdown	2.54×	2.32×	2.45×
70B tuned	55.73 _{0.56}	1.26 _{0.02}	0.86 _{0.00}
70B proxy-tuned	88.17 _{1.41}	1.79 _{0.07}	1.40 _{0.02}
Slowdown	1.58×	1.42×	1.63×

Table 12: Per-generation runtimes in three different generation settings, as described in §C.1. The column names describe the length of the prompt and the length of the generation, in that order. The mean and standard deviation per generation are reported.

C Additional Analysis

C.1 Runtime Analysis

Our goal in this analysis is to measure how proxy-tuning affects observed runtime. We measure runtime in three different settings, varying the number of tokens in the prompt and in the generation: (8-token prompt, 512-token output), (512-token prompt, 8-token output), and (8-token prompt, 8-token output). The prompt is created by repeating the word “hi” until the desired prompt length is reached. We force the length of the output to be exactly the desired output length, by suppressing the end-of-sequence token until the output length is reached, and then ending the generation.

For each setting, we greedily decode 100 outputs for the prompt, and record the clocktime of each generation. We report the results for proxy-tuned LLAMA2 and LLAMA2-CHAT at both 13B and 70B scale, to compare the runtime efficiency of proxy-tuning versus a true tuned model. We run 13B tuned and proxy-tuned models with 1 A100 GPU, and 70B models with 5 A100 GPUs. In all cases, we ensure that this is the only job running on the hardware.

Shown in Table 12, at 13B, there is a $\sim 2.4\times$ increase in runtime; at 70B, there is a $\sim 1.5\times$ increase. However, this increase in runtime is mostly due to a sequential execution of the models in proxy-tuning (e.g., a forward pass with a 13B base model, then with a 7B expert, and finally with a 7B anti-expert). In practice, proxy-tuning can be greatly accelerated by deploying on multiple GPUs in parallel that communicate with each other (e.g., through an `allreduce` operation). This way, at each decoding step, the forward passes with each model run at the same time, and then the logit scores are gathered, sampled, and distributed back to each device through the GPU communication. Our pilot implementation shows a similar runtime compared to a true tuned model (though using three GPUs instead of one).

C.2 How often, and at what position, does proxy-tuning change the prediction?

In this section, we analyze how often proxy-tuning changes the top-token prediction from the base model in instruction-tuning experiments (§3). The percentage of predictions changed is 17.3% for AlpacaFarm, 24.6% for Toxigen, 13.5% for GSM, and 18.0% for TruthfulQA (open-ended). Figure 3 shows how this value varies across positions in generation for AlpacaFarm. We see that proxy-tuning has the largest influence on the earliest tokens. The plots for other datasets have the same pattern.

This suggests that the runtime cost of proxy-tuning may be reduced through methods that efficiently select which time steps to enact proxy-tuning. We note that the simple approach of only applying proxy-tuning to the first few tokens has limited effectiveness, due to the base model’s tendency to return to endless repetition when unchecked.

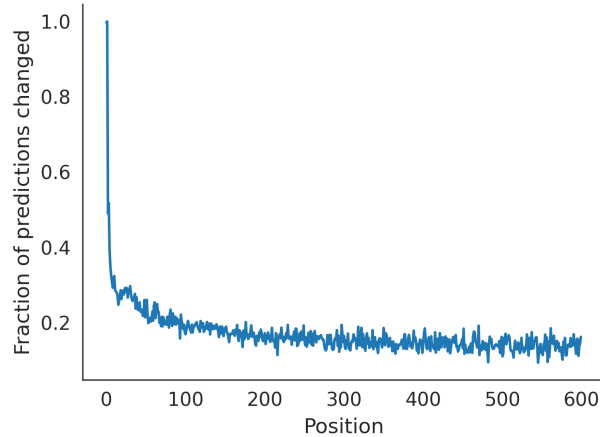


Figure 3: The fraction of top-token predictions changed by proxy-tuning (compared to the base model) at every position in the generation, for AlpacaFarm.

Hyperparameter	Assignment
Learning rate	1e-4
LoRA rank	64
LoRA alpha	16
LoRA dropout	0.1

Table 13: Hyperparameters for finetuning task-specific models with LORA. All unspecified hyperparameters are the same as those for full finetuning in Table 11.

Setting	TriviaQA	GSM
Full 7B	30h : 11m	2h : 35m
LoRA 13B	33h : 55m	3h : 49m
Slowdown	1.12×	1.48×
LoRA 70B	459h : 06m	39 hr 20 m
Slowdown	15.21×	15.23×

Table 14: **Comparison of training time** for the 7B model through full finetuning (for proxy-tuning) versus the 13B/70B model with LoRA, on the same hardware.

D Comparison with LORA for Task Finetuning

While proxy-tuning requires only black-box access to models, in this section we also compare to an efficient finetuning method available in the white-box setting, namely low-rank adaptation (LoRA; Hu et al., 2022). LoRA trains rank decomposition matrices injected into each layer of the Transformer, while freezing the rest of the model. Note that it requires access to all parameters of the base model (and the resources to load them).

We use the experimental setup of §5, the only case where we tune models ourselves rather than using off-the-shelf tuned models. We use the same training hyperparameters as used by Tulu 2 (Iverson et al., 2023) for QLoRA (Dettmers et al., 2023), shown in Table 13.

Overall, we find that even in white-box settings, proxy-tuning is a valuable option, especially when considering training efficiency.

D.1 Empirical Comparison

Shown in Table 15, LoRA’s performance depends on the task and model size. For TriviaQA, LoRA consistently outperforms even *full* finetuning by a large margin — by 6.5% at 13B, and 12.2% at 70B. On GSM, the relative effectiveness between proxy-tuning and LORA is mixed — at 13B, proxy-tuning outperforms LORA by 11.5%, while at 70B, LORA outperforms proxy-tuning by 9.1%. The finding that LORA performs closer to full finetuning with larger model size is consistent with prior work (Lester et al., 2021).

Model	TriviaQA	GSM
<i>13B</i>		
Base (untuned)	36.8	6.6
Proxy-tuned	55.9	43.9
LoRA	66.0	32.4
Full tuning	59.5	51.0
<i>70B</i>		
Base (untuned)	45.2	9.6
Proxy-tuned	62.7	53.9
LoRA	75.3	63.0
Full tuning	63.1	67.9

Table 15: Comparison between proxy-tuning and LoRA for task-specific finetuning. Note that only the two LoRA rows are new relative to Table 5.

We hypothesize that LoRA’s inconsistency across the two tasks is due to the size of the shift between pretraining and finetuning data, where a smaller shift can be more easily captured through parameter-efficient finetuning. As suggested by examples in Table 18, TriviaQA answers are short and very stylistically similar to the base model’s original predictions, whereas GSM answers are long passages with particular formatting idiosyncracies, qualitatively very far from the base model’s prediction.

D.2 Train Efficiency

We additionally report the training efficiency of LoRA and proxy-tuning (i.e., full finetuning of a 7B model) in Table 14. We see that full finetuning of a 7B expert is far more train-efficient than LoRA for a 13B or 70B model — by $1.3\times$ and $15\times$, respectively. All tuning is done on 4 A100s for a fair comparison.

E Example Generations

Example generations for each dataset studied in §3, §4, and §5 are shown in Table 16, Table 17, and Table 18, respectively. The examples are not cherry picked; for each dataset, we show the very first example in the dataset. The only exception is DS-1000, which is downsampled, so we show the first example in the sample.